

CENTRO COMUNITÁRIO DE ATENDIMENTO

CCA BOM JESUS DO CANGAÍBA

SisGESC – ERP EDUCACIONAL

Dicionário de Dados – Projeto de Banco de Dados

Projeto de Banco de Dados — Entrega Final

Professor: Clóvis Ferraro
GRUPO 1 – QUINTA (MANHÃ)

São Paulo 2026

SUMÁRIO

1 INTRODUÇÃO.....	4
2 CONVENÇÕES E LEGENDA.....	4
2.1 Padrão de Nomenclatura.....	4
2.2 Legenda de Cores das Tabelas do Dicionário	5
2.3 Tipos de Dados Utilizados	5
3 DICIONÁRIO DE DADOS POR TABELA	5
3.1 Módulo Acadêmico	5
3.1.1 tb_aluno – Cadastro Central de Alunos.....	5
3.1.2 tb_responsavel – Gestão de Responsáveis Legais.....	6
3.1.3 tb_contato_responsavel – Normalização de Contatos dos Responsáveis.....	7
3.1.4 tb_aluno_responsavel – Entidade Associativa N:N	7
3.1.5 tb_vinculo_familiar – Rastreamento de Núcleos Familiares	8
3.1.6 tb_turma – Estrutura das Turmas e Faixas Etárias.....	8
3.1.7 tb_matricula – Registro de Vínculos Acadêmicos.....	9
3.1.8 tb_lista_espera – Fila de Espera e Promoção Automática	10
3.1.9 tb_frequencia – Controle Diário de Presença.....	10
3.1.10 tb_alerta – Sinalização de Risco e Vulnerabilidade	11
3.2 Módulo de Recursos Humanos (RH).....	12
3.2.1 tb_cargo – Definição de Funções e Cargas Horárias	12
3.2.2 tb_funcionario – Cadastro de Colaboradores	12
3.2.3 tb_jornada_trabalho – Escalas Diárias de Trabalho	13
3.2.4 tb_registro_ponto – Controle de Entrada e Saída	14
3.2.5 tb_professor_turma – Alocação Docente N:N	14
3.3 Módulo Financeiro.....	15
3.3.1 tb_programa_social – Fontes de Custeio e Convênios	15
3.3.2 tb_repassse – Gestão de Recursos Recebidos	15
3.3.3 tb_categoria_gastos – Classificação de Despesas	16
3.3.4 tb_conta – Gestão Bancária e de Saldos	16
3.3.5 tb_gasto – Rastreabilidade de Recursos Públicos	17
3.3.6 tb_fatura – Ciclo de Vida de Contas a Pagar	17
3.3.7 tb_pagamento_fatura – Quitação e Conciliação Bancária.....	18
3.3.8 tb_pagamento_funcionario – Folha Mensal e Vínculo com Repasses.....	18
4 REGRAS DE NEGÓCIO	20
4.1 Regra da Lista de Espera.....	20
4.2 Desligamento ao Completar 14 Anos	21
5 ESTRATÉGIA ANALÍTICA: CAMPOS DE BI E IA	22
6 NORMALIZAÇÃO: 1FN, 2FN E 3FN.....	22

6.1 Primeira Forma Normal (1FN)	22
6.2 Segunda Forma Normal (2FN)	23
6.3 Terceira Forma Normal (3FN).....	23
7 SCRIPTS SQL COMPLETOS (DDL E DML)	23
7.1 Criação do Banco de Dados e Função fn_validar_cpf	23
7.2 Módulo Acadêmico – Criação das Tabelas	24
7.3 Módulo RH – Criação das Tabelas	26
7.4 Módulo Financeiro – Criação das Tabelas	26
7.5 Views	28
7.6 Triggers	29
7.7 Stored Procedure: sp_chamar_lista_espera	29
7.8 Dados Iniciais (INSERT).....	30
8 INTEGRAÇÃO ENTRE MÓDULOS	32
8.1 Acadêmico ↔ RH	32
8.2 Acadêmico ↔ Financeiro.....	32
8.3 RH ↔ Financeiro.....	32
9 GITHUB	33
10 DER OLAP+OLTP	34
10.1 DER OLTP – Diagrama Entidade-Relacionamento Transacional.....	35
10.2 DER SQL – Esquema Físico Implementado	36
10.3 DER OLAP – Modelo Dimensional (Data Warehouse).....	36

1 INTRODUÇÃO

Este documento apresenta o Dicionário de Dados completo do sistema SisGESC — Sistema de Gestão Educacional e Social — desenvolvido em MySQL para atender às necessidades operacionais do Centro Comunitário de Atendimento (CCA) Bom Jesus do Cangaíba, localizado na cidade de São Paulo.

A instituição é vinculada a uma entidade religiosa (igreja) e opera como organização filantrópica sem fins lucrativos, voltada ao atendimento socioeducativo de crianças e adolescentes em situação de vulnerabilidade social, oferecendo suporte pedagógico e social no contraturno escolar.

O CCA atende atualmente até 120 alunos simultaneamente, distribuídos em exatamente 3 turmas fixas com capacidade de 40 alunos cada, nos turnos matutino e vespertino. A faixa etária atendida é de 8 a 14 anos completos, sendo o desligamento automático ao atingir 14 anos uma regra central do sistema.

O banco de dados foi projetado para gerenciar informações acadêmicas, de recursos humanos e financeiras, incluindo o Número de Identificação Social (NIS), controle de frequência, jornada de trabalho dos funcionários, repasses de programas sociais e rastreabilidade financeira dos recursos públicos. O objetivo central é digitalizar e sistematizar o fluxo de registros manuais, garantindo precisão, transparência e conformidade com os requisitos dos órgãos públicos financiadores.

O projeto está estruturado em três módulos integrados:

- Módulo Acadêmico: cadastro de alunos, turmas, matrículas, frequência, lista de espera e alertas de vulnerabilidade.
- Módulo de Recursos Humanos (RH): cadastro de funcionários, cargos, jornada de trabalho, controle de ponto e alocação docente.
- Módulo Financeiro: gestão de programas sociais, repasses, gastos, contas bancárias, faturas e pagamentos de pessoal.

2 CONVENÇÕES E LEGENDA

2.1 Padrão de Nomenclatura

Todos os identificadores do banco de dados utilizam o padrão snake_case, com os seguintes prefixos obrigatórios, conforme estabelecido pelo professor orientador:

Tipo	Prefixo	Exemplo
Tabela	tb_	tb_aluno
Chave Primária	pk_	pk_aluno
Chave Estrangeira	fk_	fk_turma
Atributo Comum	(sem prefixo)	data_nascimento

2.2 Legenda de Cores das Tabelas do Dicionário

Tipo de Linha	Cor	Significado
Chave Primária (PK)	Verde-claro (#D9EAD3)	Identifica o atributo que é chave primária da tabela
Chave Estrangeira (FK)	Amarelo (#FFF2CC)	Identifica o atributo que é chave estrangeira (relacionamento)
Atributo Comum (par)	Cinza-claro (#F2F2F2)	Atributo comum em linha de índice par
Atributo Comum (ímpar)	Branco (#FFFFFF)	Atributo comum em linha de índice ímpar

2.3 Tipos de Dados Utilizados

O projeto adota os seguintes tipos de dados MySQL, respeitando as orientações do professor:

- INT — Números inteiros (PKs, FKs, contadores, idades).
- CHAR(n) — Texto de tamanho fixo (CPF com 11 dígitos, mês de referência com 7 caracteres).
- VARCHAR(n) — Texto de tamanho variável com limite explícito definido (nomes, descrições, status).
- DATE — Data no formato AAAA-MM-DD (nascimentos, matrículas, datas de pagamento).
- DATETIME — Data e hora (campos de auditoria data_criacao).
- TIME — Hora (jornada de trabalho, registros de ponto).
- DECIMAL(10,2) — Valores financeiros com precisão de duas casas decimais (salários, faturas, gastos).
- BOOLEAN — Valores lógicos 0/1 (presente, responsavel_legal, principal).
- TINYINT — Retorno de funções booleanas (fn_validar_cpf).

Tipos evitados: FLOAT (imprecisão para dinheiro) e VARCHAR sem tamanho (inconsistência de armazenamento).

3 DICIONÁRIO DE DADOS POR TABELA

3.1 Módulo Acadêmico

3.1.1 tb_aluno – Cadastro Central de Alunos

Armazena os dados cadastrais de todas as crianças e adolescentes atendidos pelo CCA. É a tabela central do módulo acadêmico, referenciada por matrículas, alertas, vínculos familiares e relacionamentos com responsáveis. O campo nome_aluno não possui restrição UNIQUE, pois o identificador natural da pessoa é o CPF (cpf_aluno). Um aluno que completa 14 anos não é removido do sistema — seu campo situacao_aluno é alterado para

"concluido", preservando todo o histórico. O campo idade é persistido no cadastro e validado por CHECK (idade >= 8 AND idade <= 14).

Atributo	Tipo de Dado	Restrição	Descrição	Observação
pk_aluno	INT	PK, AUTO_INCREMENT, NOT NULL	Identificador único do aluno	Gerado automaticamente pelo banco
nome_aluno	VARCHAR(120)	NOT NULL	Nome completo do aluno	Não possui UNIQUE; CPF é o identificador natural
nis_aluno	CHAR(11)	NOT NULL, UNIQUE	Número de Identificação Social	Somente dígitos; validado por CHECK REGEXP <code>^[0-9]{11}\$</code>
cpf_aluno	CHAR(11)	NOT NULL, UNIQUE	CPF do aluno	Validado por trigger com algoritmo de dígito verificador (RN03)
sexo	VARCHAR(10)	NOT NULL, CHECK	Sexo biológico declarado	Valores: "Masculino" "Feminino"
data_nascimento	DATE	NOT NULL	Data de nascimento do aluno	Base para cálculo de idade e controle de desligamento
idade	INT	NOT NULL, CHECK	Idade declarada no cadastro	CHECK (idade >= 8 AND idade <= 14) — implementa RN01
raca_cor	VARCHAR(20)	NOT NULL, DEFAULT "Nao declarada"	Autodeclaração de raça/cor conforme IBGE	Valores: Branca Preta Parda Amarela Indigena Nao declarada
situacao_aluno	VARCHAR(12)	NOT NULL, DEFAULT "ativo", CHECK	Status atual do aluno	ativo inativo transferido concluido — feature BI/IA
data_criacao	DATETIME	DEFAULT CURRENT_TIMESTAMP	Data e hora do registro no sistema	Auditoria; preenchido automaticamente pelo banco

3.1.2 tb_responsavel – Gestão de Responsáveis Legais

Armazena os responsáveis legais dos alunos. O CPF é utilizado como chave primária natural (PK), eliminando a necessidade de uma pk_responsavel artificial. Um responsável pode estar vinculado a múltiplos alunos (via tb_aluno_responsavel). Todos os contatos (telefones e e-mails) são normalizados na tabela tb_contato_responsavel, resolvendo a violação de 1FN.

Atributo	Tipo de Dado	Restrição	Descrição	Observação
cpf_responsavel	CHAR(11)	PK, NOT NULL, UNIQUE	CPF do responsável (chave natural)	Validado por trigger (RN03); somente dígitos
nome_responsavel	VARCHAR(120)	NOT NULL	Nome completo do responsável legal	
data_criacao	DATETIME	DEFAULT CURRENT_TIMESTAMP	Data e hora do cadastro	Auditoria automática

3.1.3 tb_contato_responsavel – Normalização de Contatos dos Responsáveis

Resolve a violação de Primeira Forma Normal (1FN): um responsável pode ter múltiplos telefones e múltiplos e-mails, sem limite de quantidade. O tipo de contato é discriminado pelo campo tipo_contato ("telefone" ou "email") e o formato é validado por CHECK com expressão regular. O campo principal identifica o contato preferencial do responsável.

Atributo	Tipo de Dado	Restrição	Descrição	Observação
pk_contato	INT	PK, AUTO_INCREMENT, NOT NULL	Identificador único do contato	
fk_responsavel	CHAR(11)	FK, NOT NULL	Referência ao responsável dono do contato	REFERENCES tb_responsavel(cpf_responsavel) ON DELETE CASCADE ON UPDATE CASCADE
tipo_contato	VARCHAR(10)	NOT NULL, CHECK	Tipo de contato cadastrado	Valores: "telefone" "email"
valor_contato	VARCHAR(120)	NOT NULL, CHECK	Número de telefone ou e-mail	Formato validado por CHECK REGEXP
principal	BOOLEAN	NOT NULL, DEFAULT 0	Indica se é o contato principal	1 = principal; 0 = secundário
data_criacao	DATETIME	DEFAULT CURRENT_TIMESTAMP	Data e hora do registro	Auditoria automática

3.1.4 tb_aluno_responsavel – Entidade Associativa N:N

Entidade associativa que implementa o relacionamento N:N entre alunos e responsáveis. A chave primária composta (fk_aluno, fk_responsavel) garante que um mesmo vínculo não seja duplicado. O campo responsavel_legal distingue o responsável formal (com guarda legal) de contatos secundários.

Atributo	Tipo de Dado	Restrição	Descrição	Observação
----------	--------------	-----------	-----------	------------

fk_aluno	INT	FK, NOT NULL	Referência ao aluno	REFERENCES tb_aluno(pk_aluno) ON DELETE CASCADE ON UPDATE CASCADE
fk_responsavel	CHAR(11)	FK, NOT NULL	Referência ao responsável	REFERENCES tb_responsavel(cpf_responsavel) ON DELETE RESTRICT ON UPDATE CASCADE
parentesco	VARCHAR(20)	NOT NULL, CHECK	Grau de parentesco ou vínculo	Pai Mae Avo Ava Tio Tia Padrasto Madrasta Tutor Responsavel Legal Outro
responsavel_legal	BOOLEAN	NOT NULL, DEFAULT 0	Indica responsabilidade legal formal	1 = responsável legal; 0 = contato secundário
data_criacao	DATETIME	DEFAULT CURRENT_TIMESTAMP	Data e hora do registro	Chave Primária Composta: (fk_aluno, fk_responsavel)

3.1.5 tb_vinculo_familiar – Rastreamento de Núcleos Familiares

Registra vínculos familiares entre dois alunos da mesma instituição. Permite rastrear núcleos familiares para fins de apoio social. A chave primária composta (fk_aluno_1, fk_aluno_2) evita duplicação de vínculos.

Atributo	Tipo de Dado	Restrição	Descrição	Observação
fk_aluno_1	INT	FK, NOT NULL	Primeiro aluno do vínculo familiar	REFERENCES tb_aluno(pk_aluno) ON DELETE CASCADE
fk_aluno_2	INT	FK, NOT NULL	Segundo aluno do vínculo familiar	REFERENCES tb_aluno(pk_aluno) ON DELETE CASCADE
tipo_vinculo	VARCHAR(10)	NOT NULL, DEFAULT "irmao", CHECK	Tipo de parentesco entre os alunos	Valores: irmao gêmeo primo outro
data_criacao	DATETIME	DEFAULT CURRENT_TIMESTAMP	Data e hora do registro	Chave Primária Composta: (fk_aluno_1, fk_aluno_2)

3.1.6 tb_turma – Estrutura das Turmas e Faixas Etárias

Define as turmas da instituição. O CCA opera exatamente 3 turmas fixas: Turma 1 (manhã, 08h–10h, 8 a 10 anos), Turma 2 (manhã, 10h–12h, 10 a 12 anos) e Turma 3 (tarde, 12h–14h, 12 a 14 anos). Cada turma comporta no máximo 40 alunos.

Atributo	Tipo de Dado	Restrição	Descrição	Observação
----------	--------------	-----------	-----------	------------

pk_turma	INT	PK, AUTO_INCREMENT, NOT NULL	Identificador único da turma	Gerado automaticamente pelo banco
Atributo	Tipo de Dado	Restrição	Descrição	Observação
nome_turma	VARCHAR(50)	NOT NULL, UNIQUE(nome_turma, ano_letivo)	Nome da turma	Ex: "Turma 1", "Turma 2", "Turma 3"
turno	VARCHAR(10)	NOT NULL, CHECK	Turno de funcionamento	Valores: "Manha" "Tarde"
faixa_etaria_inicio	INT	NOT NULL	Idade mínima permitida na turma (anos)	Vinculado a RN01 e RN09
faixa_etaria_fim	INT	NOT NULL	Idade máxima permitida na turma (anos)	Vinculado a RN01 e RN06
capacidade_max	INT	NOT NULL, DEFAULT 40, CHECK > 0	Número máximo de alunos na turma	RN09: Cada turma = 40 alunos; total instituição = 120
ano_letivo	INT	NOT NULL	Ano letivo de referência da turma	Compõe UNIQUE composto com nome_turma
status_turma	VARCHAR(10)	NOT NULL, DEFAULT "ativa", CHECK	Situação atual da turma	Valores: "ativa" "encerrada"
data_criacao	DATETIME	DEFAULT CURRENT_TIMESTAMP	Data e hora do registro	Auditoria automática

3.1.7 tb_matricula – Registro de Vínculos Acadêmicos

Registra as matrículas dos alunos em turmas. Um aluno só pode ter UMA matrícula ativa por vez (RN02), garantido pelo trigger trg_matricula_unica_insert. A tabela preserva histórico: matrículas encerradas ficam com situacao_matricula = "cancelada" ou "concluída".

Atributo	Tipo de Dado	Restrição	Descrição	Observação
pk_matricula	INT	PK, AUTO_INCREMENT, NOT NULL	Identificador único da matrícula	
fk_aluno	INT	FK, NOT NULL	Referência ao aluno matriculado	REFERENCES tb_aluno(pk_aluno) ON DELETE CASCADE ON UPDATE CASCADE
fk_turma	INT	FK, NOT NULL	Referência à turma de destino	REFERENCES tb_turma(pk_turma) ON DELETE RESTRICT ON UPDATE CASCADE

data_matricula	DATE	NOT NULL	Data em que a matrícula foi realizada	
situacao_matricula	VARCHAR(10)	NOT NULL, DEFAULT "ativa", CHECK	Status atual da matrícula	ativa cancelada concluída — feature BI/IA
data_encerramento	DATE	NULL	Data de encerramento da matrícula	NULL enquanto ativa; preenchido ao cancelar/concluir
Atributo	Tipo de Dado	Restrição	Descrição	Observação
data_criacao	DATETIME	DEFAULT CURRENT_TIMESTAMP	Data e hora do registro	Auditoria automática

3.1.8 tb_lista_espera – Fila de Espera e Promoção Automática

Gerencia a fila de espera de alunos para quando as turmas estão lotadas. A promoção do próximo da fila é feita chamando a stored procedure sp_chamar_lista_espera(fk_turma), que seleciona o aluno aguardando há mais tempo e cria automaticamente a matrícula.

Atributo	Tipo de Dado	Restrição	Descrição	Observação
pk_espera	INT	PK, AUTO_INCREMENT, NOT NULL	Identificador único do registro na lista	
fk_aluno	INT	FK, NOT NULL	Referência ao aluno em espera	REFERENCES tb_aluno(pk_aluno) ON DELETE CASCADE
fk_turma	INT	FK, NOT NULL	Referência à turma desejada	REFERENCES tb_turma(pk_turma) ON DELETE CASCADE
data_solicitacao	DATE	NOT NULL	Data em que o aluno entrou na lista	Feature BI/IA: mede tempo de espera médio
status_espera	VARCHAR(15)	NOT NULL, DEFAULT "aguardando", CHECK	Status do aluno na lista	aguardando chamado matriculado cancelado
data_chamada	DATE	NULL	Data em que o aluno foi chamado para matrícula	Preenchido pela procedure sp_chamar_lista_espera
data_criacao	DATETIME	DEFAULT CURRENT_TIMESTAMP	Data e hora do registro	Auditoria automática

3.1.9 tb_frequencia – Controle Diário de Presença

Registra a frequência diária dos alunos por matrícula. Cada linha representa um aluno em um dia específico. A constraint UNIQUE(fk_matricula, data_aula) garante que um aluno não possa ter dois registros de frequência no mesmo dia (RN08).

Atributo	Tipo de Dado	Restrição	Descrição	Observação
pk_frequencia	INT	PK, AUTO_INCREMENT, NOT NULL	Identificador único do registro de frequência	
fk_matricula	INT	FK, NOT NULL	Referência à matrícula do aluno	REFERENCES tb_matricula(pk_matricula) ON DELETE CASCADE
data_aula	DATE	NOT NULL, UNIQUE(fk_matricula, data_aula)	Data da aula registrada	1 registro por matrícula por dia — implementa RN08
presente	BOOLEAN	NOT NULL, DEFAULT 1	Indica se o aluno estava presente	1 = presente; 0 = ausente — feature BI/IA
Atributo	Tipo de Dado	Restrição	Descrição	Observação
motivo_falta	VARCHAR(100)	NULL	Justificativa da falta	Preenchido somente em ausências quando informado
data_criacao	DATETIME	DEFAULT CURRENT_TIMESTAMP	Data e hora do registro	Auditoria automática

3.1.10 tb_alerta – Sinalização de Risco e Vulnerabilidade

Registra alertas de risco gerados por funcionários sobre alunos em situação de vulnerabilidade. Integra os módulos Acadêmico e RH (fk_aluno + fk_funcionario). O campo nivel_risco é ordinal (Baixo → Médio → Alto → Crítico) e serve como feature preditiva para modelos de IA.

Atributo	Tipo de Dado	Restrição	Descrição	Observação
pk_alerta	INT	PK, AUTO_INCREMENT, NOT NULL	Identificador único do alerta	
fk_aluno	INT	FK, NOT NULL	Aluno sobre o qual o alerta foi gerado	REFERENCES tb_aluno(pk_aluno) ON DELETE RESTRICT
fk_funcionario	CHAR(11)	FK, NOT NULL	Funcionário que registrou o alerta	REFERENCES tb_funcionario(cpf_funcionario) ON DELETE RESTRICT
tipo_alerta	VARCHAR(25)	NOT NULL, CHECK	Categoria do alerta	"Frequencia Critica" "Vulnerabilidade" "Evasao Iminente"
nivel_risco	VARCHAR(10)	NOT NULL, DEFAULT "Medio", CHECK	Grau de urgência do alerta	"Baixo" "Medio" "Alto" "Critico" — feature preditiva BI/IA
descricao_alerta	VARCHAR(200)	NOT NULL	Descrição detalhada da situação de risco	Texto livre para o funcionário registrar o contexto

data_alerta	DATE	NOT NULL	Data em que o alerta foi gerado	
status_alerta	VARCHAR(20)	NOT NULL, DEFAULT "Aberto", CHECK	Status atual do acompanhamento	"Aberto" "Em Acompanhamento" "Resolvido"
data_resolucao	DATE	NULL	Data de resolução do caso	Preenchido ao alterar status para "Resolvido"
data_criacao	DATETIME	DEFAULT CURRENT_TIMESTAMP	Data e hora do registro	Auditoria automática

3.2 Módulo de Recursos Humanos (RH)

3.2.1 tb_cargo – Definição de Funções e Cargas Horárias

Cadastro dos cargos existentes na instituição, separado de tb_funcionario para eliminar dependência transitória e atingir a Terceira Forma Normal (3FN). Cargos existentes: Professora, Coordenadora, Assistente Social, Auxiliar Administrativo, Chefe de Unidade.

Atributo	Tipo de Dado	Restrição	Descrição	Observação
pk_cargo	INT	PK, AUTO_INCREMENT, NOT NULL	Identificador único do cargo	
nome_cargo	VARCHAR(60)	NOT NULL, UNIQUE	Nome do cargo	Ex: "Professora", "Coordenadora", "Assistente Social"
descricao_cargo	VARCHAR(200)	NULL	Descrição das atribuições do cargo	Campo opcional
carga_horaria_padrao	INT	NOT NULL, CHECK > 0	Carga horária semanal padrão em horas	Ex: Professora = 30h; demais = 40h
data_criacao	DATETIME	DEFAULT CURRENT_TIMESTAMP	Data e hora do registro	Auditoria automática

3.2.2 tb_funcionario – Cadastro de Colaboradores

Armazena os dados dos funcionários da instituição. O CPF é utilizado como chave primária natural. O cargo é referenciado via FK em tb_cargo (3FN). Professoras Luciana e Michelle têm jornada das 08h às 17h e são alocadas nas três turmas via tb_professor_turma.

Atributo	Tipo de Dado	Restrição	Descrição	Observação
cpf_funcionario	CHAR(11)	PK, NOT NULL, UNIQUE	CPF do funcionário (chave natural)	Validado por trigger com algoritmo de dígito verificador (RN03)

fk_cargo	INT	FK, NOT NULL	Cargo exercido pelo funcionário	REFERENCES tb_cargo(pk_cargo) ON DELETE RESTRICT ON UPDATE CASCADE
nome_funcionario	VARCHAR(120)	NOT NULL	Nome completo do funcionário	
data_admissao	DATE	NOT NULL	Data de admissão na instituição	
tipo_vinculo	VARCHAR(15)	NOT NULL, DEFAULT "CLT", CHECK	Tipo de vínculo empregatício	"CLT" "Estatutario" "Voluntario"
salario	DECIMAL(10,2)	NOT NULL, CHECK > 0	Salário mensal do funcionário	DECIMAL(10,2) para precisão
Atributo	Tipo de Dado	Restrição	Descrição	Observação
				financeira — vinculado a RN08
carga_horaria_semanal	INT	NOT NULL, CHECK > 0	Carga horária semanal efetiva em horas	
status_funcionario	VARCHAR(15)	NOT NULL, DEFAULT "ativo", CHECK	Status atual do funcionário	"ativo" "afastado" "desligado"
data_criacao	DATETIME	DEFAULT CURRENT_TIMESTAMP	Data e hora do registro	Auditoria automática

3.2.3 tb_jornada_trabalho – Escalas Diárias de Trabalho

Define a jornada diária de trabalho de cada funcionário por dia da semana. Professoras Luciana e Michelle têm jornada das 08h00 às 17h00, de segunda a sexta.

Atributo	Tipo de Dado	Restrição	Descrição	Observação
pk_jornada	INT	PK, AUTO_INCREMENT, NOT NULL	Identificador único da jornada	
fk_funcionario	CHAR(11)	FK, NOT NULL	Funcionário ao qual a jornada pertence	REFERENCES tb_funcionario(cpf_funcionario) ON DELETE CASCADE
dia_semana	VARCHAR(15)	NOT NULL, CHECK	Dia da semana da jornada	"Segunda" "Terca" "Quarta" "Quinta" "Sexta"
hora_entrada	TIME	NOT NULL	Horário de entrada	Ex: 08:00
hora_saida	TIME	NOT NULL	Horário de saída	Ex: 17:00

data_criacao	DATETIME	DEFAULT CURRENT_TIMESTAMP	Data e hora do registro	Auditoria automática
--------------	----------	------------------------------	-------------------------	----------------------

3.2.4 tb_registro_ponto – Controle de Entrada e Saída

Registra o ponto diário dos funcionários, com horário de entrada e saída. A constraint UNIQUE(fk_funcionario, data_registro) impede registro duplicado para o mesmo funcionário no mesmo dia.

Atributo	Tipo de Dado	Restrição	Descrição	Observação
pk_ponto	INT	PK, AUTO_INCREMENT, NOT NULL	Identificador único do registro de ponto	
fk_funcionario	CHAR(11)	FK, NOT NULL	Funcionário que realizou o registro	REFERENCES tb_funcionario(cpf_funcionario) ON DELETE CASCADE
Atributo	Tipo de Dado	Restrição	Descrição	Observação
data_registro	DATE	NOT NULL, UNIQUE(fk_funcionario, data_registro)	Data do registro de ponto	UNIQUE composto impede ponto duplicado no mesmo dia
hora_entrada	TIME	NULL	Hora de entrada registrada	Pode ser NULL em casos de registro parcial
hora_saida	TIME	NULL	Hora de saída registrada	NULL quando funcionário ainda não saiu
data_criacao	DATETIME	DEFAULT CURRENT_TIMESTAMP	Data e hora do registro no sistema	Auditoria automática

3.2.5 tb_professor_turma – Alocação Docente N:N

Entidade associativa N:N entre professores (funcionários) e turmas. As professoras Luciana Silva e Michelle Souza são alocadas nas três turmas simultaneamente. Chave primária composta: (fk_funcionario, fk_turma).

Atributo	Tipo de Dado	Restrição	Descrição	Observação
fk_funcionario	CHAR(11)	FK, NOT NULL	Funcionário que é professor da turma	REFERENCES tb_funcionario(cpf_funcionario) ON DELETE RESTRICT
fk_turma	INT	FK, NOT NULL	Turma em que o professor atua	REFERENCES tb_turma(pk_turma) ON DELETE RESTRICT
data_criacao	DATETIME	DEFAULT CURRENT_TIMESTAMP	Data e hora do vínculo registrado	Auditoria automática

3.3 Módulo Financeiro

3.3.1 tb_programa_social – Fontes de Custeio e Convênios

Registra os programas sociais (convênios, fundos municipais) que são as fontes de custeio financeiro do CCA. Todos os repasses recebidos e pagamentos de funcionários são rastreados até um programa social, garantindo prestação de contas aos órgãos financiadores.

Atributo	Tipo de Dado	Restrição	Descrição	Observação
pk_programa	INT	PK, AUTO_INCREMENT, NOT NULL	Identificador único do programa social	
nome_programa	VARCHAR(100)	NOT NULL, UNIQUE	Nome oficial do programa ou convênio	Ex: "Convênio SMDHC 2025"
descricao	VARCHAR(200)	NULL	Descrição do programa e seu objetivo	Campo opcional
data_criacao	DATETIME	DEFAULT CURRENT_TIMESTAMP	Data e hora do registro	Auditoria automática

3.3.2 tb_repassse – Gestão de Recursos Recebidos

Registra cada repasse financeiro recebido de um programa social. É o ponto de entrada dos recursos que financiam gastos e salários. O campo mes_referencia segue o formato "AAAA-MM", validado por CHECK com REGEXP. A view vw_saldo_repassse calcula em tempo real o saldo disponível de cada repasse.

Atributo	Tipo de Dado	Restrição	Descrição	Observação
pk_repassse	INT	PK, AUTO_INCREMENT, NOT NULL	Identificador único do repasse	
fk_programa	INT	FK, NOT NULL	Programa social de origem do repasse	REFERENCES tb_programa_social(pk_programa) ON DELETE RESTRICT
data_repassse	DATE	NOT NULL	Data em que o repasse foi recebido	
valor_repassse	DECIMAL(12,2)	NOT NULL, CHECK > 0	Valor monetário do repasse	DECIMAL(12,2) para precisão; integrado à view vw_saldo_repassse
mes_referencia	CHAR(7)	NOT NULL, CHECK REGEXP	Mês de competência do repasse	Formato "AAAA-MM" — validado por CHECK REGEXP <code>^[0-9]{4}-[09]{2}\$</code>

descricao	VARCHAR(200)	NULL	Descrição complementar do repasse	Campo opcional
data_criacao	DATETIME	DEFAULT CURRENT_TIMESTAMP	Data e hora do registro	Auditoria automática

3.3.3 tb_categoria_gastos – Classificação de Despesas

Categoriza os tipos de despesas da instituição. Categorias existentes: Alimentação, Material Pedagógico, Manutenção, Pagamento de Pessoal, Transporte, Contas Fixas.

Atributo	Tipo de Dado	Restrição	Descrição	Observação
pk_categoria	INT	PK, AUTO_INCREMENT, NOT NULL	Identificador único da categoria	
nome_categoria	VARCHAR(60)	NOT NULL, UNIQUE	Nome da categoria de gasto	Ex: "Alimentacao", "Manutencao", "Pagamento de Pessoal"
descricao	VARCHAR(150)	NULL	Descrição detalhada da categoria	Campo opcional
data_criacao	DATETIME	DEFAULT CURRENT_TIMESTAMP	Data e hora do registro	Auditoria automática

3.3.4 tb_conta – Gestão Bancária e de Saldos

Registra as contas bancárias da instituição utilizadas para pagamentos. O campo numero_conta possui restrição UNIQUE, garantindo unicidade de cadastro.

Atributo	Tipo de Dado	Restrição	Descrição	Observação
pk_conta	INT	PK, AUTO_INCREMENT, NOT NULL	Identificador único da conta	
nome_conta	VARCHAR(100)	NOT NULL	Nome/apelido da conta bancária	Ex: "Conta Corrente CCA"
banco	VARCHAR(60)	NOT NULL	Nome do banco onde a conta está registrada	Ex: "Banco do Brasil"
agencia	CHAR(10)	NOT NULL	Número da agência bancária	Ex: "1234-5"
numero_conta	VARCHAR(20)	NOT NULL, UNIQUE	Número completo da conta	UNIQUE garante cadastro único
saldo	DECIMAL(12,2)	NOT NULL, DEFAULT 0.00	Saldo disponível na conta	DECIMAL(12,2) para precisão monetária
data_criacao	DATETIME	DEFAULT CURRENT_TIMESTAMP	Data e hora do registro	Auditoria automática

3.3.5 tb_gasto – Rastreabilidade de Recursos Públicos

Registra os gastos operacionais diretamente vinculados a um repasse específico e a uma categoria. Garante rastreabilidade completa do uso dos recursos públicos. O campo nota_fiscal suporta auditoria e prestação de contas aos órgãos financiadores.

Atributo	Tipo de Dado	Restrição	Descrição	Observação
pk_gasto	INT	PK, AUTO_INCREMENT, NOT NULL	Identificador único do gasto	
fk_repassse	INT	FK, NOT NULL	Repassse que financia este gasto	REFERENCES tb_repassse(pk_repassse) ON DELETE RESTRICT ON UPDATE CASCADE
fk_categoria	INT	FK, NOT NULL	Categoria do gasto	REFERENCES tb_categoria_gastos(pk_categoria) ON DELETE RESTRICT
data_gasto	DATE	NOT NULL	Data em que o gasto ocorreu	
valor_gasto	DECIMAL(10,2)	NOT NULL, CHECK > 0	Valor gasto	DECIMAL(10,2); CHECK implementa RN06
descricao	VARCHAR(200)	NULL	Descrição do gasto	Campo opcional
nota_fiscal	VARCHAR(50)	NULL	Número ou código da nota fiscal	Suporta auditoria e prestação de contas
data_criacao	DATETIME	DEFAULT CURRENT_TIMESTAMP	Data e hora do registro	Auditoria automática

3.3.6 tb_fatura – Ciclo de Vida de Contas a Pagar

Registra faturas a pagar (contas fixas como luz, água, internet). Separada de tb_gasto por ter ciclo de vida próprio, com status de acompanhamento independente ("pendente" → "paga" → "vencida").

Atributo	Tipo de Dado	Restrição	Descrição	Observação
pk_fatura	INT	PK, AUTO_INCREMENT, NOT NULL	Identificador único da fatura	
fk_categoria	INT	FK, NOT NULL	Categoria da despesa a que se refere	REFERENCES tb_categoria_gastos(pk_categoria) ON DELETE RESTRICT

descricao	VARCHAR(200)	NOT NULL	Descrição da fatura ou serviço	Ex: "Conta de luz marco/2025"
valor_fatura	DECIMAL(10,2)	NOT NULL, CHECK > 0	Valor total da fatura	DECIMAL(10,2) para precisão monetária
data_vencimento	DATE	NOT NULL	Data de vencimento da fatura	
status_fatura	VARCHAR(10)	NOT NULL, DEFAULT "pendente", CHECK	Status de pagamento da fatura	"pendente" "paga" "vencida"
data_criacao	DATETIME	DEFAULT CURRENT_TIMESTAMP	Data e hora do registro	Auditoria automática

3.3.7 tb_pagamento_fatura – Quitação e Conciliação Bancária

Registra o pagamento de uma fatura, vinculando a conta bancária utilizada. O campo valor_pago registra o valor efetivamente pago, podendo diferir do valor_fatura.

Atributo	Tipo de Dado	Restrição	Descrição	Observação
pk_pgto_fatura	INT	PK, AUTO_INCREMENT, NOT NULL	Identificador único do pagamento da fatura	
fk_fatura	INT	FK, NOT NULL	Fatura que está sendo quitada	REFERENCES tb_fatura(pk_fatura) ON DELETE RESTRICT
fk_conta	INT	FK, NOT NULL	Conta bancária de onde o pagamento foi debitado	REFERENCES tb_conta(pk_conta) ON DELETE RESTRICT
data_pagamento	DATE	NOT NULL	Data em que o pagamento foi efetuado	
valor_pago	DECIMAL(10,2)	NOT NULL, CHECK > 0	Valor efetivamente pago	DECIMAL(10,2) para precisão monetária
data_criacao	DATETIME	DEFAULT CURRENT_TIMESTAMP	Data e hora do registro	Auditoria automática

3.3.8 tb_pagamento_funcionario – Folha Mensal e Vínculo com Repasses

Registra o pagamento mensal dos funcionários, vinculando obrigatoriamente o repasse que o financia (RN04). O UNIQUE(fk_funcionario, mes_referencia) impede pagamento duplicado para o mesmo funcionário no mesmo mês (RN07).

Atributo	Tipo de Dado	Restrição	Descrição	Observação
pk_pagamento	INT	PK, AUTO_INCREMENT, NOT NULL	Identificador único do pagamento	
fk_funcionario	CHAR(11)	FK, NOT NULL, UNIQUE(fk_funcionario, mes_referencia)	Funcionário beneficiado pelo pagamento	REFERENCES tb_funcionario(cpf_funcionario) ON DELETE RESTRICT
fk_repassse	INT	FK, NOT NULL	Repassse que financia o salário deste mês	REFERENCES tb_repassse(pk_repassse) ON DELETE RESTRICT — chave de integração RH+Financeiro (RN04 e RN08)
mes_referencia	CHAR(7)	NOT NULL, CHECK REGEXP	Mês de competência do pagamento	Formato "AAAA-MM"; UNIQUE por funcionário: 1 pagamento/mês
valor_pago	DECIMAL(10,2)	NOT NULL, CHECK > 0	Valor pago ao funcionário no mês	DECIMAL(10,2); deve corresponder ao salário em tb_funcionario
data_pagamento	DATE	NOT NULL	Data em que o pagamento foi realizado	
status_pagamento	VARCHAR(10)	NOT NULL, DEFAULT "pendente", CHECK	Status do pagamento	"pendente" "pago"
Atributo	Tipo de Dado	Restrição	Descrição	Observação
data_criacao	DATETIME	DEFAULT CURRENT_TIMESTAMP	Data e hora do registro	Auditoria automática

4 REGRAS DE NEGÓCIO

O sistema implementa 10 regras de negócio (RN01 a RN10), sendo 4 delas complexas e integrando dois módulos diferentes. As regras estão implementadas por meio de CHECK constraints, triggers e stored procedures.

ID	Módulos	Descrição
RN01	Acadêmico	O aluno deve ter entre 8 e 14 anos completos no momento do cadastro. Implementado por CHECK (idade >= 8 AND idade <= 14) na tabela tb_aluno.
RN02	Acadêmico	Um aluno não pode ter mais de uma matrícula ativa simultaneamente. Garantido pelo trigger trg_matricula_unica_insert.
RN03	Acadêmico + RH + Financeiro (COMPLEXA)	O CPF deve ser matematicamente válido segundo o algoritmo oficial de dígito verificador. Implementado pela função fn_validar_cpf e por 6 triggers nas tabelas tb_aluno, tb_responsavel e tb_funcionario.
RN04	RH + Financeiro (COMPLEXA)	Todo pagamento de funcionário deve estar obrigatoriamente vinculado ao repasse do programa social que o financia. Implementado por fk_repasse NOT NULL em tb_pagamento_funcionario.
RN05	RH + Acadêmico (COMPLEXA)	Todo alerta de risco social deve conectar obrigatoriamente um aluno a um funcionário responsável pelo acompanhamento. Implementado por fk_aluno + fk_funcionario NOT NULL em tb_alerta.
RN06	Financeiro	O valor de qualquer repasse e de qualquer gasto deve ser sempre maior que zero. Implementado por CHECK (valor_repasse > 0) em tb_repasse e CHECK (valor_gasto > 0) em tb_gasto.
RN07	RH	Um funcionário não pode receber dois pagamentos no mesmo mês de referência. Implementado por UNIQUE (fk_funcionario, mes_referencia) em tb_pagamento_funcionario.
RN08	Acadêmico + Financeiro (COMPLEXA)	O registro de frequência deve ser único por aluno por dia letivo. Implementado por UNIQUE (fk_matricula, data_aula) em tb_frequencia.
RN09	Acadêmico	Cada turma não pode ultrapassar 40 alunos ativos simultaneamente. Implementado pelo trigger trg_capacidade_turma_insert.
RN10	Acadêmico	A instituição não pode ultrapassar 120 alunos ativos no total (3 salas × 40 alunos). Implementado pelo trigger trg_limite_instituicao.

4.1 Regra da Lista de Espera

Quando uma turma está cheia (capacidade_max atingida), o aluno é inserido em tb_lista_espera com status_espera = "aguardando". Ao cancelar ou concluir uma matrícula, a aplicação deve chamar a stored procedure sp_chamar_lista_espera(fk_turma), que seleciona automaticamente o próximo aluno aguardando há mais tempo e cria a matrícula dele.

```
-- 1. Encerrar a matrícula existente
UPDATE tb_matricula SET situacao_matricula = 'cancelada',
data_encerramento = CURDATE() WHERE pk_matricula = X;
-- 2. Chamar a lista de espera para aquela turma
CALL sp_chamar_lista_espera(fk_turma);
```

4.2 Desligamento ao Completar 14 Anos

Ao completar 14 anos, o aluno não é deletado do banco de dados. Seu campo `situacao_aluno` é alterado para "concluído" e o campo `situacao_matricula` da matrícula ativa é alterado para "concluída", com `data_encerramento` preenchida. O histórico completo permanece preservado para fins de rastreabilidade e análises futuras de BI/IA.

5 ESTRATÉGIA ANALÍTICA: CAMPOS DE BI E IA

Os campos abaixo foram identificados como estratégicos para análises de Business Intelligence (BI) e para treinamento de modelos preditivos de evasão escolar. Objetivo principal: Previsão de Evasão Escolar — identificar antecipadamente alunos em risco de abandono para que a equipe social possa intervir preventivamente.

Campo	Uso em BI / IA
tb_frequencia.presente	Série temporal de presença/ausência por aluno — principal feature de previsão de evasão. Taxa de presença abaixo de 75% é indicador crítico de risco.
tb_alerta.nivel_risco	Grau de urgência ordinal (Baixo → Médio → Alto → Crítico) — feature preditiva categórica para modelos de classificação de risco.
tb_alerta.tipo_alerta	Sinalização categórica de risco ("Frequência Crítica", "Vulnerabilidade", "Evasão Iminente") — permite identificar padrões de risco por categoria.
tb_aluno.situacao_aluno	Status atual do aluno (ativo/inativo/transferido/concluído) — histórico de desfechos para treino de modelos supervisionados de evasão.
tb_matricula.situacao_matricula	Histórico de cancelamentos e conclusões — target (variável alvo) de modelos preditivos de evasão. "Cancelada" antecipadamente indica evasão.
tb_aluno.data_nascimento	Cálculo dinâmico de faixa etária — feature demográfica para segmentação por idade e análise de sazonalidade.
tb_lista_espera.data_solicitacao	Tempo de espera na fila — feature de demanda e acessibilidade ao serviço, útil para planejamento de expansão.
tb_aluno.raca_cor	Análise demográfica do atendimento conforme IBGE — permite relatórios de diversidade e identificação de grupos subrepresentados.
tb_frequencia.data_aula	Base para construção de séries temporais de presença — permite calcular taxa de frequência por período e detectar padrões sazonais de faltas.

6 NORMALIZAÇÃO: 1FN, 2FN E 3FN

6.1 Primeira Forma Normal (1FN)

A 1FN exige que cada campo contenha apenas um valor atômico, sem grupos repetidos ou atributos multivalorados. O projeto resolve violações de 1FN com as seguintes decisões de projeto:

- Contatos de responsáveis (telefones e e-mails) foram extraídos para tb_contato_responsavel, eliminando o problema de múltiplos valores no mesmo campo.

- Contatos de funcionários foram extraídos para tb_contato_funcionario.
- Vínculos familiares entre alunos foram extraídos para tb_vinculo_familiar.

6.2 Segunda Forma Normal (2FN)

A 2FN exige que, em tabelas com PK composta, todos os atributos dependam da chave completa (não de parte dela). O projeto garante a 2FN nas tabelas associativas:

- Em tb_aluno_responsavel, os atributos parentesco e responsavel_legal dependem do par (fk_aluno, fk_responsavel), não de apenas um deles.
- Em tb_professor_turma, data_criacao depende do par (fk_funcionario, fk_turma).

6.3 Terceira Forma Normal (3FN)

A 3FN exige a ausência de dependências transitórias. O projeto garante a 3FN pelas seguintes decisões:

- tb_cargo é separada de tb_funcionario: carga_horaria_padrao e descricao_cargo dependem do cargo, não do funcionário.
- tb_categoria_gastos é separada de tb_gasto e tb_fatura: nome e descrição da categoria não dependem do gasto ou da fatura individual.
- tb_programa_social é separada de tb_repasse: os dados do programa não dependem do repasse específico.

7 SCRIPTS SQL COMPLETOS (DDL E DML)

Esta seção apresenta o código MySQL completo do sistema SisGESc, organizado em blocos comentados: criação do banco de dados, função de validação de CPF, tabelas por módulo, views, triggers, stored procedure e dados iniciais de exemplo.

7.1 Criação do Banco de Dados e Função fn_validar_cpf

A função fn_validar_cpf implementa o algoritmo oficial de validação de CPF. Recebe uma string de 11 dígitos, calcula os dois dígitos verificadores pela regra módulo 11 e retorna 1 (válido) ou 0 (inválido).

```

CREATE DATABASE IF NOT EXISTS sisgesc;
USE sisgesc;

DELIMITER $
CREATE FUNCTION fn_validar_cpf(p_cpf CHAR(11))
RETURNS TINYINT
DETERMINISTIC
BEGIN
    DECLARE v_soma INT DEFAULT 0;
    DECLARE v_resto INT;
    DECLARE v_i INT DEFAULT 1;
    DECLARE v_dig1 INT;
    DECLARE v_dig2 INT;
    IF LENGTH(p_cpf) <> 11 OR p_cpf REGEXP '^[^0-9]' THEN RETURN 0; END IF;
    IF p_cpf IN ('00000000000','11111111111',...) THEN RETURN 0; END IF;
    -- Calcula 1o digito verificador
    SET v_soma = 0; SET v_i = 1;
    WHILE v_i <= 9 DO
        SET v_soma = v_soma + CAST(SUBSTRING(p_cpf,v_i,1) AS UNSIGNED) * (11 - v_i);
    SET v_i = v_i + 1;
    END WHILE;
    SET v_resto = v_soma % 11;
    SET v_dig1 = IF(v_resto < 2, 0, 11 - v_resto);
    -- (calcula do 2o digito omitido por brevidade - ver script completo no
    GitHub)
    IF v_dig1 = CAST(SUBSTRING(p_cpf,10,1) AS UNSIGNED) AND v_dig2 =
    CAST(SUBSTRING(p_cpf,11,1) AS UNSIGNED)
    THEN RETURN 1; ELSE RETURN 0; END IF;
END$
DELIMITER ;

```

7.2 Módulo Acadêmico – Criação das Tabelas

```

-- Tabela 1: tb_aluno CREATE
TABLE tb_aluno (
    pk_aluno INT NOT NULL AUTO_INCREMENT, nome_aluno
    VARCHAR(120) NOT NULL, nis_aluno CHAR(11) NOT NULL,
    cpf_aluno CHAR(11) NOT NULL, sexo VARCHAR(10) NOT
    NULL, data_nascimento DATE NOT NULL, idade INT NOT
    NULL, raca_cor VARCHAR(20) NOT NULL DEFAULT 'Nao
    declarada', situacao_aluno VARCHAR(12) NOT NULL
    DEFAULT 'ativo', data_criacao DATETIME DEFAULT
    CURRENT_TIMESTAMP,
    PRIMARY KEY (pk_aluno), UNIQUE (nis_aluno), UNIQUE (cpf_aluno),
    CHECK (cpf_aluno REGEXP '^[0-9]{11}$'),
    CHECK (idade >= 8 AND idade <= 14),
    CHECK (sexo IN ('Masculino', 'Feminino')),
    CHECK (situacao_aluno IN ('ativo','inativo','transferido','concluido'))

```

```
);
```

```
-- Tabela 6: tb_turma CREATE
TABLE tb_turma (
  pk_turma INT NOT NULL AUTO_INCREMENT,  nome_turma
  VARCHAR(50) NOT NULL,  turno VARCHAR(10) NOT NULL,
  faixa_etaria_inicio INT NOT NULL,  faixa_etaria_fim
  INT NOT NULL,  capacidade_max INT NOT NULL DEFAULT
  40,  ano_letivo INT NOT NULL,  status_turma
  VARCHAR(10) NOT NULL DEFAULT 'ativa',  data_criacao
  DATETIME DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (pk_turma), UNIQUE (nome_turma, ano_letivo),
  CHECK (turno IN ('Manha', 'Tarde')),
  CHECK (capacidade_max > 0) );
```

```
-- Tabela 7: tb_matricula (RN02: Trigger impede mais de uma matricula ativa)
CREATE TABLE tb_matricula (
  pk_matricula INT NOT NULL AUTO_INCREMENT,  fk_aluno INT
  NOT NULL,  fk_turma INT NOT NULL,  data_matricula DATE
  NOT NULL,  situacao_matricula VARCHAR(10) NOT NULL
  DEFAULT 'ativa',  data_encerramento DATE,  data_criacao
  DATETIME DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (pk_matricula),
  FOREIGN KEY (fk_aluno) REFERENCES tb_aluno(pk_aluno) ON DELETE CASCADE ON
  UPDATE CASCADE,
  FOREIGN KEY (fk_turma) REFERENCES tb_turma(pk_turma) ON DELETE RESTRICT ON
  UPDATE CASCADE,
  CHECK (situacao_matricula IN ('ativa','cancelada','concluida')) );
```

```
-- Tabela 9: tb_frequencia (RN08: UNIQUE impede dois registros no mesmo dia)
CREATE TABLE tb_frequencia (
  pk_frequencia INT NOT NULL AUTO_INCREMENT,
  fk_matricula INT NOT NULL,  data_aula DATE NOT
  NULL,  presente BOOLEAN NOT NULL DEFAULT 1,
  motivo_falta VARCHAR(100),  data_criacao DATETIME
  DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (pk_frequencia),
  UNIQUE (fk_matricula, data_aula),
  FOREIGN KEY (fk_matricula) REFERENCES tb_matricula(pk_matricula) ON DELETE
  CASCADE
);
```

7.3 Módulo RH – Criação das Tabelas

```
-- Tabela 11: tb_funcionario CREATE TABLE tb_funcionario (
cpf_funcionario CHAR(11) NOT NULL,  fk_cargo INT NOT
NULL,  nome_funcionario VARCHAR(120) NOT NULL,
data_admissao DATE NOT NULL,  tipo_vinculo VARCHAR(15)
NOT NULL DEFAULT 'CLT',  salario DECIMAL(10,2) NOT NULL,
carga_horaria_semanal INT NOT NULL,  status_funcionario
VARCHAR(15) NOT NULL DEFAULT 'ativo',
  data_criacao DATETIME DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (cpf_funcionario),
  FOREIGN KEY (fk_cargo) REFERENCES tb_cargo(pk_cargo) ON DELETE RESTRICT ON
UPDATE CASCADE,

CHECK (salario > 0), CHECK (carga_horaria_semanal > 0) );
```

```
-- Tabela 14: tb_professor_turma (N:N: RH + Academico -
RN07) CREATE TABLE tb_professor_turma (  fk_funcionario
CHAR(11) NOT NULL,  fk_turma INT NOT NULL,  data_criacao
DATETIME DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (fk_funcionario, fk_turma),
  FOREIGN KEY (fk_funcionario) REFERENCES tb_funcionario(cpf_funcionario) ON
DELETE RESTRICT,
  FOREIGN KEY (fk_turma) REFERENCES tb_turma(pk_turma) ON DELETE RESTRICT );
```

7.4 Módulo Financeiro – Criação das Tabelas

```
CREATE TABLE tb_repassse (
  pk_repassse INT NOT NULL AUTO_INCREMENT,
  fk_programa INT NOT NULL,  data_repassse DATE NOT
NULL,  valor_repassse DECIMAL(12,2) NOT NULL,
mes_referencia CHAR(7) NOT NULL,  descricao
VARCHAR(200),  data_criacao DATETIME DEFAULT
CURRENT_TIMESTAMP,
  PRIMARY KEY (pk_repassse),
  FOREIGN KEY (fk_programa) REFERENCES tb_programa_social(pk_programa) ON DELETE
RESTRICT,
  CHECK (valor_repassse > 0),
  CHECK (mes_referencia REGEXP '^[0-9]{4}-[0-9]{2}$')
);
```

```
-- tb_pagamento_funcionario (RH + Financeiro – COMPLEXA)
-- RN04: fk_repasse NOT NULL garante rastreabilidade.
-- RN07: UNIQUE impede pagamento duplo no mesmo mes. CREATE
TABLE tb_pagamento_funcionario (  pk_pagamento INT NOT
NULL AUTO_INCREMENT,  fk_funcionario CHAR(11) NOT NULL,
fk_repasse INT NOT NULL,  mes_referencia CHAR(7) NOT NULL,
valor_pago DECIMAL(10,2) NOT NULL,  data_pagamento DATE
NOT NULL,  status_pagamento VARCHAR(10) NOT NULL DEFAULT
'pendente',
  data_criacao DATETIME DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (pk_pagamento),
  UNIQUE (fk_funcionario, mes_referencia),
  FOREIGN KEY (fk_funcionario) REFERENCES tb_funcionario(cpf_funcionario) ON
DELETE RESTRICT,
  FOREIGN KEY (fk_repasse) REFERENCES tb_repasse(pk_repasse) ON DELETE
RESTRICT,  CHECK (valor_pago > 0), CHECK (status_pagamento IN
('pendente', 'pago')) );
```

7.5 Views

```
-- vw_aluno: Exibe codigo formatado (CA000001) de cada aluno
CREATE VIEW vw_aluno AS
SELECT pk_aluno, CONCAT('CA', LPAD(pk_aluno, 6, '0')) AS codigo_aluno,
nome_aluno, nis_aluno, cpf_aluno, sexo, data_nascimento, idade, raca_cor,
situacao_aluno FROM tb_aluno;
```

```
-- vw_saldo_repassse: Calcula saldo disponivel por repasse em tempo real.
CREATE VIEW vw_saldo_repassse AS
SELECT r.pk_repassse, r.mes_referencia, p.nome_programa, r.valor_repassse,
```

```

    IFNULL((SELECT SUM(g.valor_gasto) FROM tb_gasto g WHERE g.fk_repassse =
r.pk_repassse), 0) AS total_gastos,
    IFNULL((SELECT SUM(pf.valor_pago) FROM tb_pagamento_funcionario pf WHERE
pf.fk_repassse = r.pk_repassse), 0) AS total_pagamentos,
    r.valor_repassse - IFNULL(...gastos...) - IFNULL(...pagamentos...) AS
saldo_disponivel
FROM tb_repassse r JOIN tb_programa_social p ON p.pk_programa = r.fk_programa;

```

7.6 Triggers

```

-- RN02: Impede que um aluno tenha mais de uma matricula ativa.
CREATE TRIGGER trg_matricula_unica_insert
BEFORE INSERT ON tb_matricula FOR EACH ROW BEGIN
    IF NEW.situacao_matricula = 'ativa' AND (
        SELECT COUNT(*) FROM tb_matricula WHERE fk_aluno = NEW.fk_aluno AND
situacao_matricula = 'ativa'
    ) > 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Aluno ja possui matricula ativa.';
    END IF;
END$

```

```

-- RN09: Impede matricula quando turma atingiu 40 alunos.
CREATE TRIGGER trg_capacidade_turma_insert
BEFORE INSERT ON tb_matricula FOR EACH ROW
BEGIN
    DECLARE v_cap_max INT; DECLARE v_matriculados INT;
    SELECT capacidade_max INTO v_cap_max FROM tb_turma WHERE pk_turma =
NEW.fk_turma;
    SELECT COUNT(*) INTO v_matriculados FROM tb_matricula WHERE fk_turma =
NEW.fk_turma AND situacao_matricula = 'ativa';
    IF NEW.situacao_matricula = 'ativa' AND v_matriculados >= v_cap_max THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Turma ja atingiu a capacidade
maxima de 40 alunos.';
    END IF;
END$

```

```

-- RN10: Impede matricula quando instituicao ja atingiu 120 alunos ativos.
CREATE TRIGGER trg_limite_instituicao
BEFORE INSERT ON tb_matricula FOR EACH ROW
BEGIN
    DECLARE v_total INT;
    SELECT COUNT(*) INTO v_total FROM tb_matricula WHERE situacao_matricula =
'ativa';
    IF NEW.situacao_matricula = 'ativa' AND v_total >= 120 THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Limite total da instituicao
atingido (120 alunos ativos).';
    END IF;
END$

```

7.7 Stored Procedure: sp_chamar_lista_espera

Esta procedure substitui o trigger que existia em versões anteriores. O MySQL não permite INSERT na mesma tabela (tb_matricula) dentro de um trigger AFTER UPDATE ON

tb_matricula. Por isso, a lógica de promoção da lista de espera foi movida para uma procedure que a aplicação chama manualmente após encerrar qualquer matrícula.

```
CREATE PROCEDURE sp_chamar_lista_espera(IN p_fk_turma INT)
BEGIN
    DECLARE v_proximo_aluno INT;
    DECLARE v_espera_id INT;

    SELECT pk_espera, fk_aluno INTO v_espera_id, v_proximo_aluno
    FROM tb_lista_espera WHERE fk_turma = p_fk_turma AND status_espera =
    'aguardando'
    ORDER BY data_solicitacao LIMIT 1;
    IF v_proximo_aluno IS NOT NULL THEN
        INSERT INTO tb_matricula (fk_aluno, fk_turma, data_matricula,
        situacao_matricula)
        VALUES (v_proximo_aluno, p_fk_turma, CURDATE(), 'ativa');
        UPDATE tb_lista_espera SET status_espera = 'matriculado', data_chamada =
        CURDATE()
        WHERE pk_espera = v_espera_id;
    END IF;
END$
```

7.8 Dados Iniciais (INSERT)

```
-- 3 turmas fixas
INSERT INTO tb_turma (nome_turma, turno, faixa_etaria_inicio,
faixa_etaria_fim, capacidade_max, ano_letivo, status_turma) VALUES ('Turma
1', 'Manha', 8, 10, 40, 2025, 'ativa'),
('Turma 2', 'Manha', 10, 12, 40, 2025, 'ativa'),
('Turma 3', 'Tarde', 12, 14, 40, 2025, 'ativa');

-- Funcionarios
INSERT INTO tb_funcionario (cpf_funcionario, fk_cargo, nome_funcionario,
data_admissao, tipo_vinculo, salario, carga_horaria_semanal, status_funcionario)
VALUES
('52345678933', 1, 'Luciana Silva', '2022-02-01', 'CLT', 3200.00, 40,
'ativo'),
('98765432029', 1, 'Michelle Souza', '2021-08-01', 'CLT', 3200.00, 40,
'ativo'),
('11122233477', 3, 'Thais Oliveira', '2020-03-15', 'Estatutario', 2800.00, 40,
'ativo'),
('44455566708', 5, 'Gilmar Costa', '2019-01-10', 'Estatutario', 4500.00, 40,
'ativo');
```



8 INTEGRAÇÃO ENTRE MÓDULOS

O SisGESC é projetado como um ERP integrado, onde os três módulos se comunicam diretamente através de chaves estrangeiras e lógica de negócio:

8.1 Acadêmico ↔ RH

A tabela tb_professor_turma conecta funcionários a turmas (N:N), permitindo que Luciana e Michelle atuem nas três turmas. A tabela tb_alerta conecta alunos a funcionários, registrando quem gerou cada alerta de vulnerabilidade.

8.2 Acadêmico ↔ Financeiro

O módulo acadêmico se integra ao financeiro principalmente pela necessidade de rastreabilidade dos recursos públicos. Cada frequência pode ser vinculada ao repasse que financiou aquela aula. O número de alunos ativos impacta diretamente o valor dos convênios.

8.3 RH ↔ Financeiro

A tabela tb_pagamento_funcionario é a peça central da integração RH + Financeiro. Cada pagamento de salário obrigatoriamente referencia o repasse que o financia (RN04), garantindo que os auditores possam rastrear exatamente qual verba pública pagou cada funcionário em cada mês.

9 GITHUB

O código-fonte completo do projeto, incluindo todos os scripts SQL (DDL e DML), está disponível no repositório GitHub do grupo.

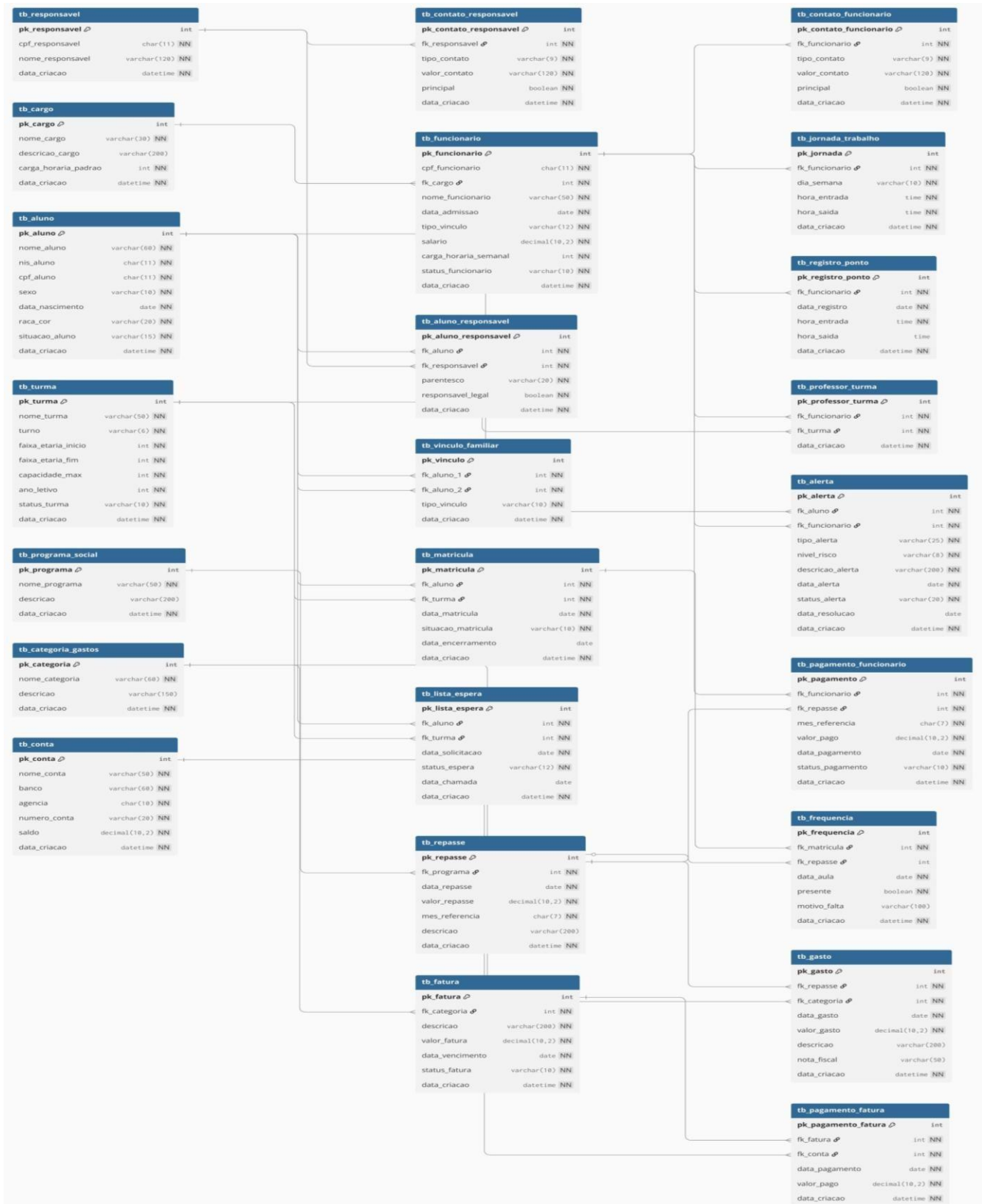
[REPOSITÓRIO](#)

10 DER OLAP+OLTP

Esta seção apresenta os Diagramas Entidade-Relacionamento dos modelos OLTP (transacional), SQL (esquema físico implementado) e OLAP (Data Warehouse dimensional) do sistema SisGESC.

10.1 DER OLTP – Diagrama Entidade-Relacionamento Transacional

O Diagrama Entidade-Relacionamento do banco de dados transacional (OLTP) representa a estrutura operacional completa do sistema SisGESC, com todas as tabelas, atributos, tipos de dados e relacionamentos utilizados no processamento do dia a dia da instituição. O modelo cobre os três módulos integrados: Acadêmico, Recursos Humanos e Financeiro.



10.2 DER SQL – Esquema Físico Implementado

O Diagrama SQL apresenta o esquema físico implementado no banco de dados MySQL, conforme gerado pelo phpMyAdmin. Evidencia os tipos de dados efetivos (INT, CHAR, VARCHAR, DECIMAL, ENUM, TINYINT, DATE, DATETIME, TIME), as chaves primárias, chaves estrangeiras, índices, triggers e views criadas no ambiente de produção do sistema SisGESC.

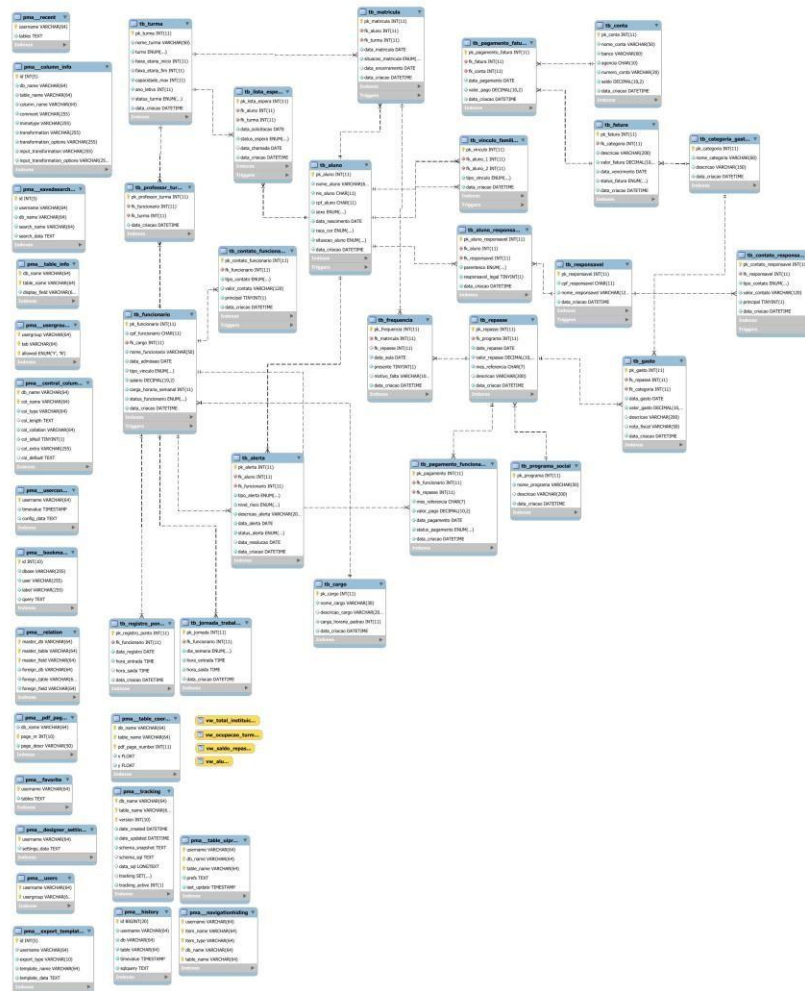


Figura 2 – DER do Esquema SQL Físico (phpMyAdmin)

10.3 DER OLAP – Modelo Dimensional (Data Warehouse)

O Diagrama Entidade-Relacionamento do Data Warehouse (OLAP) representa o modelo dimensional do sistema SisGESC, estruturado no padrão Star Schema (Esquema Estrela).

É composto por tabelas fato (ft_frequencia, ft_financeiro, ft_matricula) e dimensões (dim_tempo, dim_aluno, dim_turma, dim_funcionario, dim_programa_social, dim_categoria_gastos), utilizadas para análise gerencial e geração de relatórios de BI/IA.

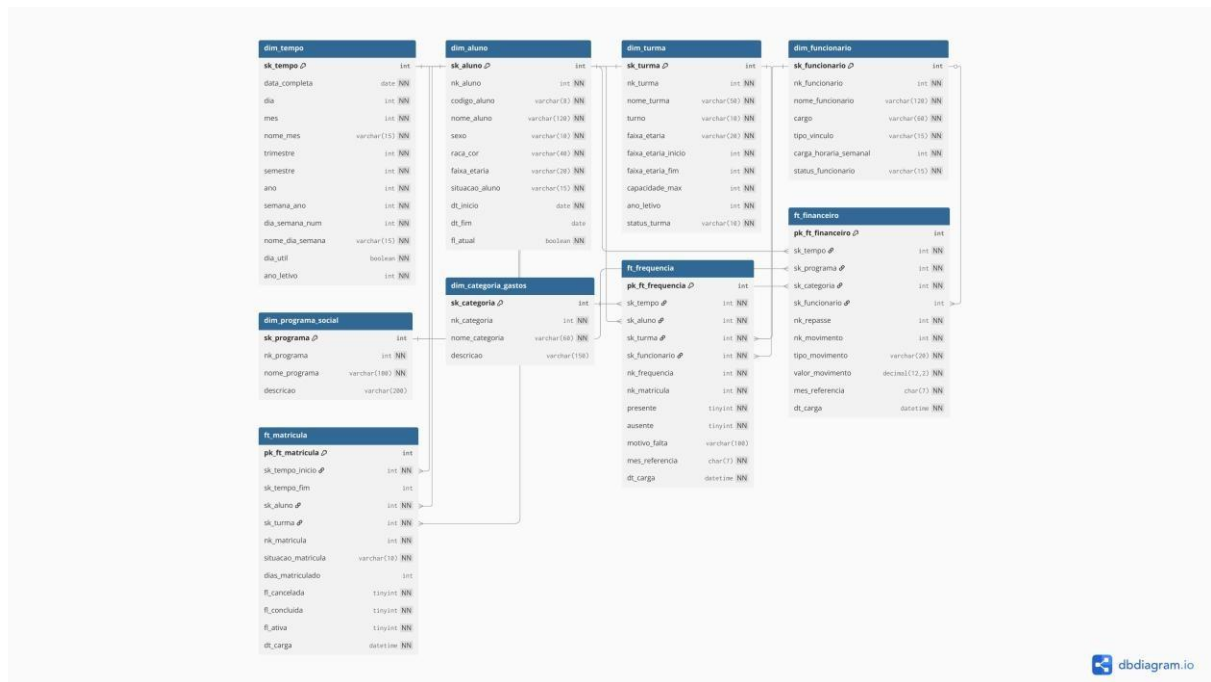


Figura 3 – DER do Data Warehouse OLAP (Star Schema)